



Homework Assignment

Week 5: String Immutability and Memory

Total Points: 100 — Due: Next Class



Name: _____

Date: _____

Important Instructions

- Work on your own, but you may ask family members for hints only if stuck
- Write your code in Python (*using Thonny or any online IDE*)
- For setting up your code files, kindly see `instructions.pdf`
- For written answers, use the spaces provided on this sheet
- Show your work for full credit!

1 Part 1: Understanding Immutability (15 points)

Time estimate: 20 minutes

1.1 Exercise 1.1: Immutable vs Mutable Prediction (15 points)

For each code snippet below, predict what will happen and explain why, (*considering string immutability*):

Code	Will it Work?	Explanation
<code>text = "hello"</code> <code>text[0] = "H"</code>	Yes/No	
<code>word = "cat"</code> <code>word = word + "s"</code>	Yes/No	
<code>name = "Ali"</code> <code>name.upper()</code> <code>print(name)</code>	Output:	Why?
<code>msg = "hi"</code> <code>msg = msg.replace("i", "ey")</code>	Yes/No	

2 Part 2: Memory Diagrams and Tracing (25 points)

Time estimate: 30 minutes

2.1 Exercise 2.1: Trace with Memory Diagrams (15 points)

For this code, draw a memory diagram showing what happens at each step. Show variables as arrows pointing to string objects in memory:

```

1 # Draw memory state after each line
2 greeting = "Hello"           # Line 1
3 name = "Sara"                # Line 2
4 greeting = greeting + " " + name # Line 3
5 original = greeting          # Line 4
6 greeting = greeting.upper()   # Line 5

```

Draw a memory diagram on a separate sheet of paper with 5 stages (*one stage after each line*), showing:

1. Stage 1: Variable 'greeting' → box with "Hello"
2. Stage 2: Add variable 'name' → box with "Sara"
3. Stage 3: Create NEW string "Hello Sara", greeting points to it
4. Stage 4: Variable 'original' points to same "Hello Sara"
5. Stage 5: Create NEW string "HELLO SARA", greeting points to it, original still points to "Hello Sara"

Final questions to answer based on your diagram:

After Line 1: _____

After Line 2: _____

After Line 3: _____ (How many strings exist now?)

After Line 4: _____

After Line 5: What does **original** contain? _____ What does **greeting** contain? _____

2.2 Exercise 2.2: Multiple Variables (10 points)

Trace this code and draw the final memory state:

```

1 text1 = "Python"
2 text2 = text1
3 text3 = "Python"
4 text2 = text2.lower()

```

Draw the memory diagram on a separate sheet of paper

From your diagram, fill in the values of each variable:

- text1 = _____
- text2 = _____
- text3 = _____

How many different string objects exist at the end? _____

3 Part 3: String Reconstruction (15 points)

Time estimate: 20 minutes

3.1 Exercise 3.1: Character Replacement (7 points)

Since we can't modify strings directly, complete this code to replace a character:

```

1  # Fix the typo in "Pyhton" to make it "Python"
2  word = "Pyhton"
3
4  # Can't do: word[2] = "t" # This would error!
5  # Instead, reconstruct the string:
6
7  # Get the part before the character to replace
8  before = word[:__] # What index?
9
10 # Get the part after the character to replace
11 after = word[__:] # What index?
12
13 # Build the new string
14 word = _____ + "t" + _____
15
16 print(word) # Should print "Python"

```

Save this code as `week5_ex3_1.py`

3.2 Exercise 3.2: Building New Strings (8 points)

Complete this program that creates modified versions of strings:

```

1  # Create different versions of a name without modifying the original
2  original_name = "hassan"
3
4  # Create a version with first letter capitalized
5  # Hint: Use slicing and concatenation
6  capitalized = _____ .upper() + _____ [__:]
7
8  # Create a version with "Mr. " added to the front
9  formal = _____ + _____
10
11 # Create a version with last letter removed
12 shortened = _____[:-__]
13
14 print("Original:", original_name)
15 print("Capitalized:", capitalized) # Should print "Hassan"
16 print("Formal:", formal) # Should print "Mr. hassan"
17 print("Shortened:", shortened) # Should print "hassa"

```

Save this code as `week5_ex3_2.py`

4 Part 4: Debug Detective (15 points)

Time estimate: 20 minutes

Each code snippet has errors related to string immutability. Find and fix ALL errors:

4.1 Snippet 1: Forgotten Save (5 points)

```
1 # This code has 2 errors - fix them all!
2 name = "  ahmed  "
3 name.strip() # Remove spaces
4 name.upper() # Make uppercase
5 print("Hello", name) # Should print "Hello AHMED"
```

Fixed version:

```
1 # Write the corrected code here:
```

4.2 Snippet 2: Direct Modification Attempt (5 points)

```
1 # Fix this code that tries to modify a string
2 password = "weak123"
3 # Make first letter uppercase
4 password[0] = password[0].upper()
5 print(password) # Should print "Weak123"
```

Fixed version:

```
1 # Write the corrected code here:
```

4.3 Snippet 3: Multiple Issues (5 points)

```
1 # This has 3 errors related to immutability
2 text = "hello world"
3 text.replace("hello", "goodbye")
4 text[6] = text[6].upper()
5 result = text
6 result.upper()
7 print(result) # Should print "GOODBYE WORLD"
```

Fixed version:

```
1 # Write the corrected code here:
```

Save all corrected versions in `week5_debug_fixed.py`

5 Part 5: Output Matching (10 points)

Time estimate: 15 minutes

5.1 Exercise 5.1: Match the Output (10 points)

Draw lines to match each code with its output. Remember: string methods return NEW strings!

Code	Output
<pre>s = "abc" t = s t = t.upper() print(s)</pre>	"ABC"
<pre>x = "hi" x.upper() print(x)</pre>	"abc"
<pre>a = "cat" b = a + "s" print(a)</pre>	"hi"
<pre>m = "DOG" n = m.lower() print(n)</pre>	"cat"
<pre>p = "yo" * 2 print(p)</pre>	"dog"
<pre>w = "END" w = w[0] + w[1:].lower() print(w)</pre>	"yoyo"
	"End"

6 Part 6: Creative Coding Challenge (15 points)

Time estimate: 30 minutes

Choose ONE of the following projects and complete it:

6.1 Option A: Password Validator

Create a program that checks and fixes passwords without modifying the original:

```

1  # password_validator.py
2  # Program that validates and suggests improved passwords
3
4  original_password = input("Enter password: ")
5
6  # Check requirements (don't modify original!)
7  has_upper = False
8  has_lower = False
9  has_number = False
10
11 # Check each character
12 for char in original_password:
13     # Add your checks here
14     pass
15
16 # Create improved version if needed
17 if not has_upper:
18     # Create new password with first letter uppercase
19     improved = -----
20 else:
21     improved = original_password
22
23 # Add more improvements...
24
25 print("Original password:", original_password)

```

```
26 print("Suggested password:", improved)
27 print("Original unchanged:", original_password) # Prove it's unchanged!
```

6.2 Option B: Text Art Generator

Create a program that generates text art patterns without modifying the original string:

```
1 # text_art.py
2 # Generate different text patterns from a word
3
4 word = input("Enter a word: ")
5
6 # Create different patterns (don't modify original!)
7 # Pattern 1: Pyramid
8 for i in range(1, len(word) + 1):
9     print(word[:i])
10
11 # Pattern 2: Reverse pyramid
12 # Your code here
13
14 # Pattern 3: Wave (alternating upper/lower)
15 wave = ""
16 for i in range(len(word)):
17     if i % 2 == 0:
18         # Add uppercase character to wave
19         pass
20     else:
21         # Add lowercase character to wave
22         pass
23
24 print("Original word still:", word)
```

Save your chosen project as `password_validator.py` or `text_art.py`

7 Part 7: Reflection Questions (5 points)

Time estimate: 10 minutes

Answer these questions in complete sentences:

1. Why do you think Python designers made strings immutable? What problems does this prevent?

2. You tried `text.upper()` and nothing seemed to happen. Explain to a friend why the text didn't change and how to fix it.

3. If strings WERE mutable (changeable), what confusing situations might occur when two variables point to the same string?

8 Bonus Section: Extra Challenges (Optional - 10 extra points)

Only attempt if you've finished everything else!

8.1 Challenge 1: Efficient String Building (5 points)

This code is inefficient because it creates many intermediate strings. Improve it:

```
1 # Inefficient version
2 result = ""
3 for letter in "hello":
4     result = result + letter.upper() # Creates 5 new strings!
5
6 # Write a more efficient version using a list and join:
```

8.2 Challenge 2: String Surgery (5 points)

Write a function that removes a character at a specific index without modifying the original:

```
1 def remove_char(text, index):
2     """Remove character at index without modifying original"""
3     # Your code here
4
5
6
7 # Test your function
8 original = "mistake"
9 fixed = remove_char(original, 5) # Remove 'k'
10 print(fixed) # Should print "mistae"
11 print(original) # Should still print "mistake"
```

Submission Checklist

Before submitting, make sure you have:

- ☐ Completed all required sections (Parts 1-7)
- ☐ Saved each code exercise in its own file:
 - week5_ex3_1.py, week5_ex3_2.py
 - week5_debug_fixed.py
 - Your chosen creative project file
- ☐ Created a folder named `week5_hw_yourname` with all files
- ☐ Tested each code file individually to ensure it runs without errors
- ☐ Included memory diagrams for Part 2 (drawn on this sheet)
- ☐ Verified that your code doesn't try to modify strings directly

- ☐ Checked that all string operations save their results
- ☐ Written your name at the top of this paper
- ☐ Answered all reflection questions
- ☐ Attempted bonus section (optional)

Module 2 Concepts Check:

Which concept was most challenging? _____

Do you understand why we can't change strings directly? Yes / No / Mostly

Time Tracking:

How long did this homework take you? _____ hours

Parent/Guardian Signature: _____

(Confirming student completed work independently)

Great job exploring string immutability!

Next week: Advanced string operations - split, join, and formatting